INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE FÍSICA Y MATEMÁTICAS

Licenciatura en Matemáticas Algorítmicas

Unidad de Aprendizaje:

Análisis Matemático

Conjunto de Cantor

Alumno:

Joel Reyes Garcia

Ciudad de México

27 de marzo de 2026

Cálculo Algorítmico del Conjunto

Cuando empecé a modelar el Conjunto de Cantor, me di cuenta rápido de que el problema real no era dibujar las líneas, sino manejar la memoria. Como la cantidad de segmentos crece con una complejidad espacial de $\mathcal{O}(2^n)$, Python se quedaba sin recursos o se volvía muy lento al calcular millones de intervalos. Por eso decidí hacer el cálculo pesado en Rust. Rust es excelente manejando la memoria de forma eficiente, lo cual era justo lo que necesitaba para este algoritmo.

Para no complicarme demasiado conectando ambos lenguajes, utilicé `maturin` y `PyO3`. Estas herramientas me permitieron escribir mi función matemática en Rust, compilarla fácilmente y luego importarla en Python como si fuera una librería normal. Así, Rust hace todo el trabajo duro de calcular las coordenadas, y Python solo se encarga de lo que hace mejor: dibujar y mostrar la interfaz al usuario.

```rust
use pyo3::prelude::*;

#[pyfunction]
fn conjunto_de_cantor(iteraciones: u32) -> PyResult<Vec<(f64, f64)>> {
    let mut intervalos = vec![(0.0, 1.0)];

    for _ in 0..iteraciones {
        let mut nuevos_intervalos = Vec::with_capacity(intervalos.len() * 2);

        for (inicio, fin) in intervalos {
            let distancia = (fin - inicio) / 3.0;

            nuevos_intervalos.push((inicio, inicio + distancia));
            nuevos_intervalos.push((fin - distancia, fin));
        }
        intervalos = nuevos_intervalos;
    }

    Ok(intervalos)
}

#[pymodule]
fn CantorLib(m: &Bound<'_, PyModule>) -> PyResult<()> {
    m.add_function(wrap_pyfunction!(conjunto_de_cantor, m)?)?;
    Ok(())
}
```

Listing 1: Cálculo del Conjunto de Cantor optimizado en memoria usando Rust

En el código, usé un tipo `u32` (entero sin signo) para las iteraciones porque, matemáticamente, no vamos a pedir iteraciones negativas, y mi computadora saturaría su memoria RAM mucho antes de llegar al límite de ese número. Lo que sí era vital era usar flotantes de 64 bits (`f64`) para guardar los intervalos.

El mayor truco de optimización que implementé fue `Vec::with_capacity`. En lugar de dejar que el programa adivinara cuánta memoria iba a necesitar y perdiera tiempo reasignando arreglos, le digo exactamente cuánto espacio reservar desde el inicio de cada paso (exactamente el doble del vector anterior). Después, solo es cuestión de tomar cada segmento, calcular la distancia dividiendo su longitud entre 3, y guardar las nuevas coordenadas de los extremos izquierdo y derecho (ver Figura 1).

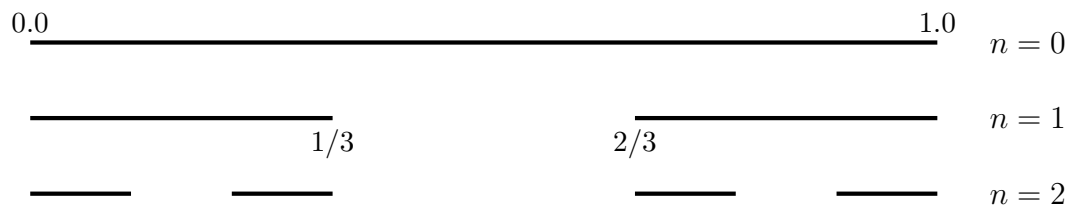

Figura 1: Representación gráfica de la división matemática programada en el ciclo de Rust.

Visualización e Interfaz

Con la matemática resuelta, el script de Python importa la librería `CantorLib.so` recién compilada. Agregué un `input` sencillo para que el usuario elija hasta qué nivel del fractal quiere profundizar. Luego, meto esos datos en arreglos de `NumPy` para manejarlos en bloque y uso `Matplotlib` para pintarlos en pantalla.

```python
import os
import sys
import time
import subprocess
import shutil
import importlib.util
import math
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.collections import LineCollection

if os.path.exists("CantorLib/Cargo.toml"):
    try:
        subprocess.check_call(
            ["cargo", "build", "--release"],
            cwd="CantorLib", stdout=subprocess.DEVNULL, stderr=subprocess.STDOUT
        )
        shutil.copy("CantorLib/target/release/libCantorLib.so", "CantorLib.so")
    except Exception as e:
```

```python
        if not os.path.exists("CantorLib.so"): sys.exit(1)

ruta_so = os.path.abspath("CantorLib.so")
spec = importlib.util.spec_from_file_location("CantorLib", ruta_so)
CantorLib = importlib.util.module_from_spec(spec)
sys.modules["CantorLib"] = CantorLib
spec.loader.exec_module(CantorLib)

ITERACIONES = int(input('Elige el numero de iteraciones: \n'))

t0 = time.time()
segs_por_nivel = []
for i in range(ITERACIONES + 1):
    inter = CantorLib.conjunto_de_cantor(i)
    segs_por_nivel.append(np.array(inter, dtype=np.float64))

print(f"Calculados {2**(ITERACIONES+1)-1:,} elementos en {time.time() - t0:.3f}s")

fig, ax = plt.subplots(figsize=(12, 8))
coleccion = LineCollection([], colors='black', linewidths=8.0, antialiased=False)
coleccion.set_capstyle('butt')
ax.add_collection(coleccion)

def actualizar_escena(event_ax=None):
    for txt in list(ax.texts):
        txt.remove()

    xlim = ax.get_xlim()
    ancho_vista = xlim[1] - xlim[0]

    if ancho_vista <= 0: return

    min_dx = ancho_vista / 1920.0

    try:
        j_max = int(math.log(1.0 / min_dx) / math.log(3.0)) + 1
    except ValueError:
        j_max = ITERACIONES

    segs_a_dibujar = []

    for i in range(ITERACIONES + 1):
        k = min(i, j_max, ITERACIONES)
        segs_k = segs_por_nivel[k]

        mask = (segs_k[:, 1] >= xlim[0]) & (segs_k[:, 0] <= xlim[1])
        visibles = segs_k[mask]

        if len(visibles) > 0:
            n_vis = len(visibles)
```

```
            y_val = -i

            bloque = np.empty((n_vis, 2, 2))
            bloque[:, 0, 0] = visibles[:, 0]
            bloque[:, 0, 1] = y_val
            bloque[:, 1, 0] = visibles[:, 1]
            bloque[:, 1, 1] = y_val

            segs_a_dibujar.append(bloque)

            if i <= j_max:
                for x0, x1 in visibles:
                    dx_real = x1 - x0
                    if dx_real > (ancho_vista * 0.15):
                        caja_inicio = dict(boxstyle="round,pad=0.2", facecolor="#f4f8fa", edgecolor="#0055aa", alpha=0.9)
                        caja_fin = dict(boxstyle="round,pad=0.2", facecolor="#faf4f4", edgecolor="#aa0000", alpha=0.9)

                        ax.annotate(f"{x0:.16g}", xy=(x0, y_val), xytext=(5, 18),
                                    textcoords="offset points", ha='left', va='bottom',
                                    fontsize=9, family='monospace', color='#003366',
                                    bbox=caja_inicio, arrowprops=dict(arrowstyle="->", color="#0055aa", shrinkB=4))

                        ax.annotate(f"{x1:.16g}", xy=(x1, y_val), xytext=(-5, -18),
                                    textcoords="offset points", ha='right', va='top',
                                    fontsize=9, family='monospace', color='#660000',
                                    bbox=caja_fin, arrowprops=dict(arrowstyle="->", color="#aa0000", shrinkB=4))

    if segs_a_dibujar:
        coleccion.set_segments(np.vstack(segs_a_dibujar))
    else:
        coleccion.set_segments([])

    if event_ax:
        ax.figure.canvas.draw_idle()

ax.callbacks.connect('xlim_changed', actualizar_escena)

ax.set_xlim(-0.05, 1.05)
ax.set_ylim(-ITERACIONES - 1, 1)
ax.axis('off')
plt.title(f"Conjunto de Cantor - {ITERACIONES} Iteraciones\n", fontsize=14)
```

```
plt.tight_layout()

actualizar_escena()

plt.show()
```

Listing 2: Interfaz de ejecución y renderizado vectorial en Python

Resultados y Rendimiento

Los resultados confirman que delegar el cálculo algorítmico a Rust fue una excelente decisión. El programa calcula $2,097,151$ elementos en apenas 0.633 segundos. Estos cálculos los realicé pidiendo 20 iteraciones al sistema, que es el número recomendado para ver la complejidad del fractal sin quedarme sin memoria RAM.

Si miramos la gráfica sin acercarnos (Figura 2), parece que los segmentos simplemente se unen en líneas sólidas debido a los píxeles de la pantalla. Pero la precisión matemática está ahí. Al hacer zoom en la gráfica, el sistema vuelve a calcular qué partes estamos viendo y renderiza las coordenadas sin ningún tipo de retraso, probando que no se perdió ni un solo decimal en las profundidades del fractal (Figura 3).

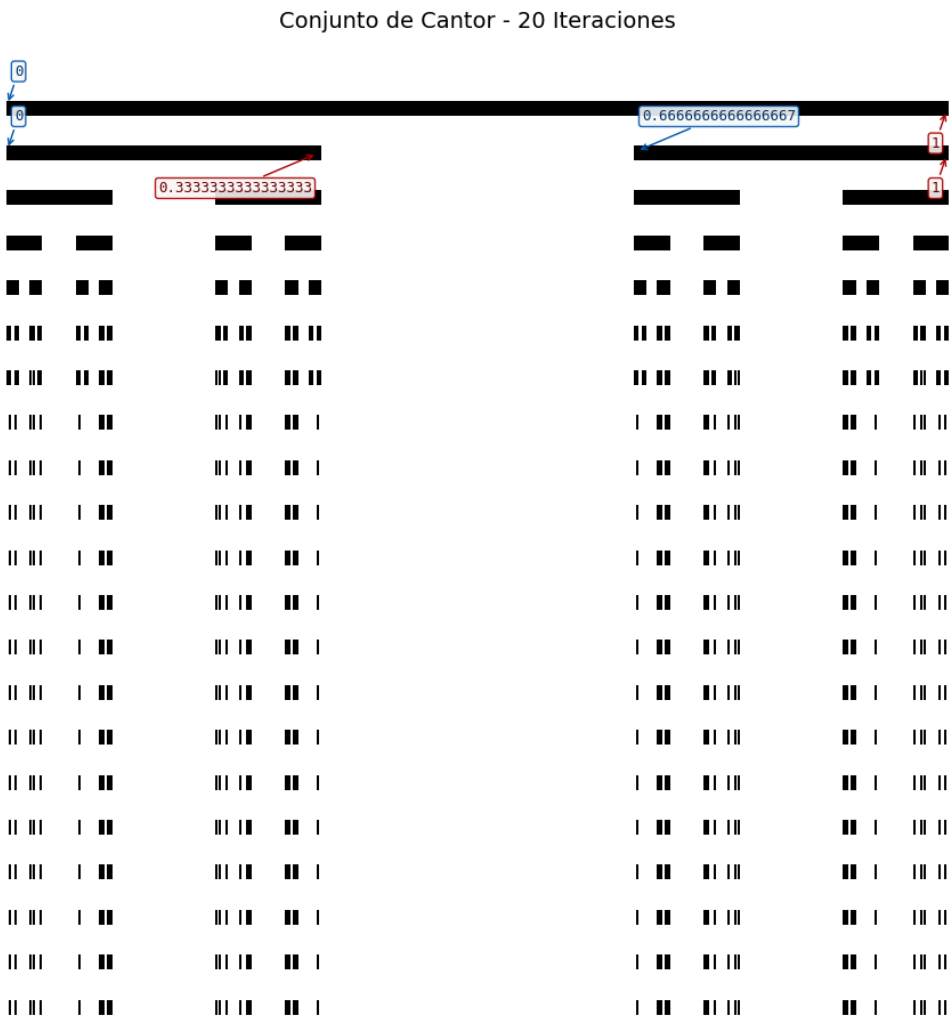

Figura 2: Vista general del Conjunto de Cantor. En pocos milisegundos se despliegan más de dos millones de segmentos en pantalla.

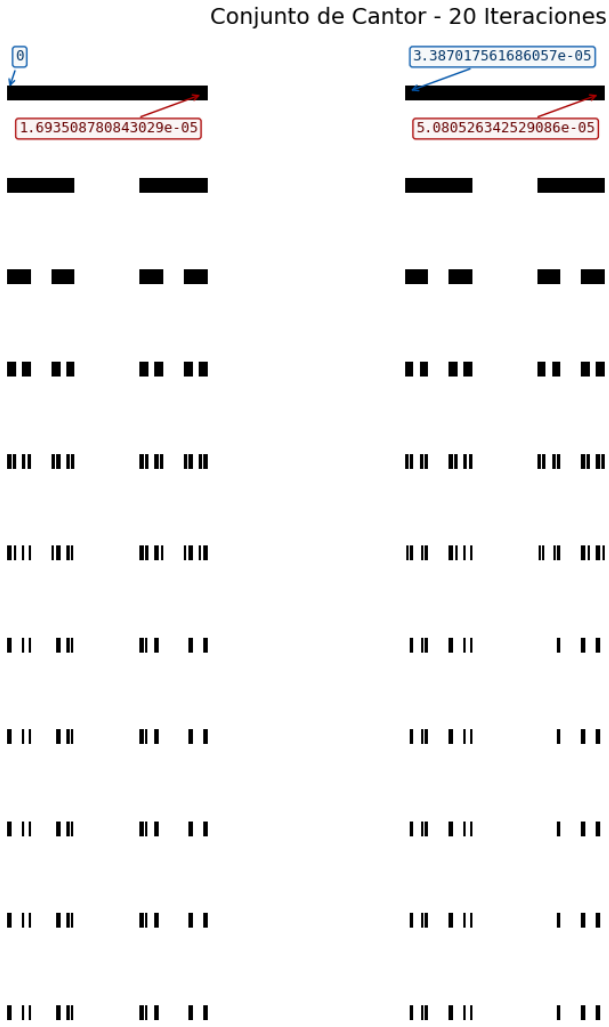

Figura 3: Detalle de las coordenadas al hacer zoom. Los subsegmentos se definen perfectamente gracias al cálculo en f64.

Repositorio del Proyecto

El código fuente completo de esta implementación está disponible en GitHub:

https://github.com/Jowl0/Conjunto_De_Cantor